Supporting Data and Code: Reproducibility Documentation

AI foundation model choice and false positive variation in Anti Money Laundering transaction screening

Companion to the *Discover Artificial Intelligence* submission

1. Purpose and scope

This document describes the supporting data-and-code archive that accompanies the manuscript (this repository). The archive is a self-contained, offline, deterministic reproduction: from a single command it regenerates every reported number, table, and figure and verifies each result by re-running the analysis and comparing byte for byte. No network access, no API keys, and no paid model calls are required. The permanent citable version is the Zenodo archive of this repository (MIT licence for code, CC-BY-4.0 for data and text), minted at archive time.

The study uses only *synthetic* transaction data. A strict wall separates two phases: an offline *analysis* that reproduces every claim from frozen inputs (documented here), and a live *capture* phase that originally queried commercial model APIs and is retained for transparency only (Section 8). Because commercial models change over time, the capture is deliberately not part of the reproducible core; the frozen model-response corpus it produced is shipped and hash-verified.

2. Quick start

```
pip install -r code/requirements.txt
bash reproduce.sh
```

Or, for a byte-identical run in the pinned container:

```
cd environment
docker build -t repro . && docker run --rm repro
```

The archive is a plain source release, so there is no container to press a button in: `bashreproduce.sh` is the whole procedure.

Expected result: the integrity check passes, the full analysis regenerates the committed outputs, the second independent run is byte-identical (SHA-256 of every result file matches), and the run prints `##REPRODUCIBLE` and exits 0. Total runtime is well under a minute on a standard laptop.

3. Archive layout

`reproduce.sh`	one-command offline reproduction + integrity + byte-identical check
`code/requirements.txt`	pinned Python dependencies (analysis only)
`analysis/`	all analysis code (see Section 5)

capture/	live data-collection harness (reference only; not run here)
tests/	pytest sanity checks on the reproduced numbers
environment/Dockerfile	pinned image for a byte-identical run (no network)
docs/	frozen pre-registration, amendment, pivot, decisions log, data-collection note, model manifest
data/	all inputs
battery/full.jsonl	PRIMARY 600-case synthetic battery (pilot.jsonl is the seeded subset)
capture/runs_full_*.csv	frozen model-response corpus, 10 files
frozen/*.freeze.json	per-file SHA-256 receipts for each capture CSV
configs/	battery.yaml, models.yaml, grid.yaml
manifest/battery_manifest.json	content hash + counts for the battery
README.md	data provenance + schema
results/	all generated outputs (see Section 6)

4. Data (inputs and metadata)

4.1 Frozen inputs

Every input the pipeline reads is a frozen artifact under data/. The analysis is a pure, seeded function of these files.

File	Rows	Unit of observation / key fields
battery/full.jsonl (PRIMARY)	600	one synthetic case = a small transaction sub-network; 300 legitimate hard negatives + 300 suspicious across eight FATF/SAML-D typologies. Fields: case_id, label, typology, difficulty, stratum, benign_pattern, n_nodes, n_edges, serialized (raw node/edge ledger), content_sha256
capture/runs_full_<model>_<variant>.csv	12,120	one row = one model call. The confirmatory design is 5 models × 2 prompt variants × 2 seeds × 600 cases = 12,000 cells; the frozen CSVs hold 12,120 rows, the extra 120 being retried calls after transient API errors. Fields: model_key, api_model, provider, prompt_variant, seed, case_index, case_id, label, typology, decision (flag / no_flag / ERROR), rationale, input_tokens, output_tokens, cost_usd, raw_response
configs/battery.yaml	—	battery specification (typologies, counts, difficulty spread, seed 20260715)
configs/models.yaml	—	model registry: provider, API snapshot/alias, list price per 1M tokens, tier
configs/grid.yaml	—	capture grid (models × prompt variants × seeds)

The primary panel is the frozen response corpus in `data/capture/`. Each per-case screening decision is reduced from its valid (non-ERROR) replicate calls by the pre-registered modal-vote rule (decision D4): on legitimate cases a 2–2 tie is not counted as a false positive (strict majority required); on suspicious cases a 2–2 tie is counted as caught.

4.2 Provenance and integrity

`data/frozen/*.freeze.json` records the SHA-256 of each of the ten capture CSVs, and `data/manifest/battery_manifest.json` records the content hash and case counts of the battery. `docs/` holds the frozen pre-registration and its amendment and pivot, so any silent change to an input or to the design is detected before the analysis runs (Section 7). `docs/data_collection.md` documents how the corpus was captured; `docs/model_manifest.md` and `docs/probe_receipts.json` record the served-model identifiers.

4.3 Data sources

All analysis inputs are synthetic and self-contained: the battery is generated deterministically (seed `20260715`) from typologies anchored to the FATF catalogue and to established synthetic-AML data structures (SAML-D; the AMLSim lineage). No external, proprietary, or personal data are used or distributed, and no download is required to reproduce any result. The original capture queried the OpenAI and Google model APIs; those responses are frozen in `data/capture/` and are the only trace of the live phase the reproduction consumes.

5. Code (what it consumes and runs)

5.1 Analysis scripts

All code is in `analysis/`. Each module is deterministic and writes only the outputs listed. Path resolution is handled centrally in `io_paths.py`: the repository is flat, so `analysis/`, `data/` and `results/` all sit beside each other and nothing hard-codes a location.

Script	Purpose	Consumes	Produces
`build_battery.py`	Regenerate the 600-case battery from the fixed seed and verify its SHA-256	`configs/battery.yaml`	`battery/full.jsonl` (verified)
`loader.py`	Load the frozen capture corpus and configs	`capture/*.csv`, `configs/`	in-memory panel
`miss.py`	Reduce replicate calls to a per-(model, case) modal decision (D4 rule)	capture rows	miss/flag table
`stats.py`	Wilson score intervals; cluster bootstrap over the eight typology strata (2,000 draws, seed 4242)	rates	confidence intervals
`classification.py`	Per-model false-positive and miss rates; Cochran's Q; McNemar exact (Bonferroni); classification metrics; prompt sensitivity	`capture/*.csv`	`classification_metrics.*`, `stats_tests.json`, `prompt_sensitivity.*`

Script	Purpose	Consumes	Produces
`altrisk.py`	Primary analysis: false-positive spread, cross-model divergence, per-typology miss, prompt flip, correlated-miss null	miss table, capture rows	`pivot_claims.json`
`baselines.py`	FATF rules baseline and a supervised baseline (5-fold out-of-fold)	`battery/full.jsonl`	`baselines.json`
`alertvolume.py`	Alert-volume projection at 0.1% prevalence over 1M tx/day	per-model rates	`alert_volume.json`
`reproduce.py`	Orchestrator: verify integrity, run the analysis, write the result tables and summary	frozen data + all modules	`results/*`
`replication_check.py`	Re-run the analysis in a second output directory and compare the SHA-256 of every result	`results/`	`replication_check.md`, `output_hashes.json`

`schema.py` defines the constrained-JSON screening schema; `io_paths.py` resolves input/output locations. The modules under `capture/` (Section 8) are not part of the reproduction.

5.2 Pipeline order

`reproduce.sh` runs the modules in dependency order. All outputs are timestamp-free, so re-runs are byte-identical:

```
verify_data (battery hash + capture freeze receipts)
  -> altrisk -> baselines -> alertvolume -> classification
  -> table1/table2 + metrics_summary
  -> replication_check (byte-identical re-run)
```

6. Results (generated outputs)

Running the pipeline regenerates the following under `results/`. In a clean checkout these already exist; the reproduction overwrites them and confirms they are unchanged.

Output	Contents
`pivot_claims.json`	every reported number with its 95% confidence interval and the configuration hash that produced it (the single source of truth for in-text numbers)
`classification_metrics.{json,csv}`	per-model precision, recall, specificity, F1, balanced accuracy, MCC, and false-positive / miss rates with Wilson intervals
`stats_tests.json`	Cochran's Q and the pairwise McNemar exact tests (Bonferroni-corrected)
`baselines.json`	rules and supervised baseline false-positive and miss rates
`alert_volume.json`	projected daily alert workload and precision per screener
`prompt_sensitivity.{json,csv}`	per-model false-positive rate under each prompt variant and the decision-flip rate

Output	Contents
table1_operating_points.{csv,md}, table2_alert_volume.{csv,md}	the manuscript result tables
metrics_summary.md	human-readable summary of the headline numbers
data_integrity.json, output_hashes.json	input-hash verification and output-hash record
replication_check.md	the PASS/FAIL byte-identical audit
figures/fig1_... -- fig5_....pdf	the five manuscript display items (static; excluded from the byte-identical check)

7. Reproduction and verification

`reproduce.sh` performs the following stages and exits non-zero if any fails:

1. **Integrity.** Regenerates the 600-case battery from seed `20260715` and confirms its SHA-256 matches `data/manifest/battery_manifest.json`; confirms each of the ten capture CSVs matches its `data/frozen/*.freeze.json` receipt. (The freeze step that produced those receipts refuses any capture file with an ERROR rate above 2%; the observed rate is 0.08%, i.e. 10 of 12,120 rows.)
2. **Regeneration.** Runs the full analysis (Section 5.2) from the frozen inputs only.
3. **Byte-identical re-run.** `replication_check.py` re-runs the whole analysis in a second, isolated output directory and confirms the SHA-256 of every result file is identical.

Determinism. Analysis seed `4242`; the cluster bootstrap uses 2,000 fixed-seed resamples over the eight typology strata, so its intervals are exactly reproducible. The reproduction makes no network calls.

Environment. The pinned analysis environment is `numpy==2.2.6`, `scikit-learn==1.7.2`, `scipy==1.18.0`, `pyyaml==6.0.3`, and `pydantic==2.13.4` (with `pytest==8.3.5` for the optional test suite), on Python 3.12. Exact pins are in `code/requirements.txt`; `environment/Dockerfile` builds the same environment for a byte-identical run on any machine. The figures are shipped as static artifacts and are excluded from the byte-identical check; regenerating them is optional and needs `matplotlib`, which is not part of the pinned analysis dependencies.

8. Live data collection (not part of the reproduction)

`capture/` documents how the model-response corpus was originally collected: `orchestrator.py` drives the capture grid; `agent.py` issues the constrained-JSON screening calls; `probe.py` pins the served-model identifier; `ledger.py` and `check_limits.py` enforce a hard spend cap; `freeze.py` writes the SHA-256 freeze receipts; `secrets.py` loads API keys. This phase requires internet access, OpenAI and Google API keys, and incurs cost, and is intentionally excluded from the reproducible core: all reported results derive only from the frozen captures in `data/capture/`.

9. Licensing and data availability

Code and data are released under the MIT Licence (see `LICENSE`). All data are synthetic; no proprietary, personal, or confidential data are used or distributed. The permanent citable archive is the Zenodo deposit of this repository, which reproduces every reported number, table, and figure offline, deterministically, and at zero cost.